ACS-2947-002/003
Lab 6
Due by Tuesday, March 11, 11:59 PM

Instructions

- The goal of this lab is to understand: 1) the implementation and use of the comparable and comparator interfaces, and 2) the implementation of Iterable and Iterator interfaces. Although the essence of learning about comparator in this course is for our Priority Queue ADT, this lab does not involve implementation of priority queues. Assignment 3 builds on this and involves priority queues**.**
- Submit your `.java` files (together in a `Lab6.zip` file) via Nexus.
- Include your name and student number as a comment in every file.

Task

1. Create a `Month` class which includes:
   a. Instance variables that store the following information:
      - Month name
      - An integer representing the month of the year
      - Total number of days in a month
      - Number of working days in a month
   b. A full-arg constructor
   c. `freeDays()` method that calculates and returns an `int` of the total number of non-working days in a month. **Include any other necessary methods.**
   d. Implementation of the Comparable interfaces' method (with natural ordering that is by the integer value for the month of the year)

2. Implement a concrete Comparator class called `MonthComparator` for an alternate comparison of `Month` objects.
   - You will need to implement the `java.util.Comparator` interface
   - The comparison rule should be as follows:
     - The month with the ***lower*** number of non-working days is *ordered before* the other month.
     - If two months have the same number of non-working days, the one with the ***higher*** number of days is *ordered before* the other month.
     - If they have the same number of days then do an ***alphabetical order*****ing** of the names of the months. *Note that the String class implements the Comparable interface.*
   - Consider the best use of if/else statements in your implementation.

3. Implement a **generic** default Comparator class (*see notes L12_)* which implements the comparator interface but uses the natural ordering of the object type.

4. In your `Lab6_Driver` class, fill an `ArrayList` (`java.util.ArrayList`) with `Month` objects. *Include all the months of the year and their respective number representation, total number of days, and number of working days. Use the following values for the number of working days in each month from January to December, respectively: 21, 20, 23, 21, 22, 22, 21, 23, 22, 21, 22, 22*

5. Using the `ArrayList` of `Month` objects, sort the months using `Collections.Sort`. Display the list of months after sorting by their natural order in two ways:
   a) With no comparator specified
   b) With the default comparator specified

6. Sort the months in the order specified by the `MonthComparator` and display the months.

Sample output
(The *number of days and number of working days are displayed next to each month, respectively*):

```
Natural ordering (With No Comparator):
January 31 21
February 28 20
March 31 23
April 30 21
May 31 22
June 30 22
July 31 21
August 31 23
September 30 22
October 31 21
November 30 22
December 31 22

Natural ordering (With Default Comparator):
January 31 21
February 28 20
March 31 23
April 30 21
May 31 22
June 30 22
July 31 21
August 31 23
September 30 22
October 31 21
November 30 22
December 31 22
```

```
Ordering by MonthComparator:
August 31 23
March 31 23
June 30 22
November 30 22
September 30 22
February 28 20
December 31 22
May 31 22
April 30 21
January 31 21
July 31 21
October 31 21
```

7. Complete the implementation of the provided `ArrayQueue` class that uses dynamic arrays (initial array capacity must be 4) and implements the provided `Queue` interface. Note that the queue interface extends `Iterable`. Include the following in the class:
   - Private `resize` method
   - `enqueue` and `toString` methods
   - `next` method in the `ArrayIterator` inner class
   - `clear` method that removes all elements from the underlying array in a O(1) run time

8. In your `Lab6_Driver` class create a static method named `printBinaryNumbers` that declares an instance of the `ArrayQueue` from above with the default constructor and use it to print the binary representation of numbers from 1 to n, where n is the parameter.
   Your queue MUST hold string values, and you will create the binary numbers as strings using the available Queue methods.
   Note that except for the input *n*, you should only be working with `String` data type i.e. you cannot use `Integer.toBinaryString` or any other shortcuts.

   Demonstrate your program with *n* = 16.

   Sample output

   ```
   1 10 11 100 101 110 111 1000 1001 1010 1011 1100 1101 1110 1111
   10000
   ```

   Hint:
   Over n iterations:
   Get the value of the first item
   Append it with a “0” then add it to the end of the queue
   Append it with a “1” then add it to the end of the queue
   Dequeue the first item and print

9. Include the following lines of code at the end of your `printBinaryNumbers` method to print the elements remaining in your queue (after the binary numbers are printed), clear the queue and print the resulting state of the queue using the toString method. *Change 'queueInstance' to your queue variable name.*

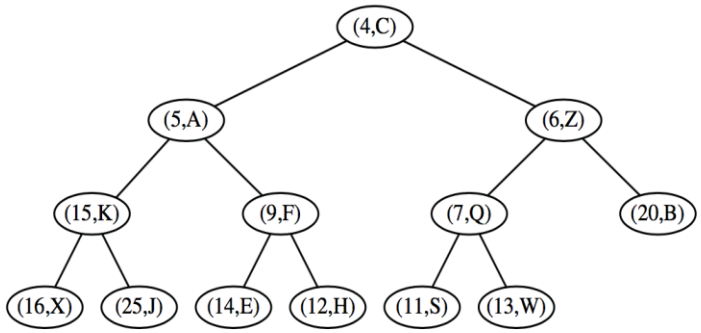
```
for (String element: queueInstance) {
   System.out.print(element + " ");
}
queueInstance.clear();
System.out.println(queueInstance);
```

Submission

Submit your **`Lab6.zip`** file that includes all the following files (`Lab6_Driver.java`, `Month.java`, `DefaultComparator.java`, `MonthComparator.java`, `Queue.java`, `ArrayQueue.java`) via **Nexus**.

EXTRA WORK: Do not submit

A Binary Tree with height *h* is a ***complete*** binary tree if levels 0, 1, 2, ..., $h - 1$ of *T* have the maximal number of nodes possible (namely, level *i* has $2i$ nodes, for $0 \le i \le h - 1$) and the remaining nodes at level *h* reside in the leftmost possible positions at that level. For example, the tree below is a complete binary tree.



Considering our LinkedBinaryTree implementation, create a method `pathToLastNode` that prints the direction (a series of "Left" and "Right") from the root to the last node in a complete binary tree. The last node is considered as the rightmode node on the deepest level of the tree. For example, for the tree above, the method should output, `Right, Left, Right` as the path to reach `(13,W)`.

Assume that the method is always applied on a complete binary tree.